

# Hi3519DV500 开发板 2GB DDR 内存重新分配指南

## 1. 背景

Hi3519DV500 开发板物理 DDR 为 2GB（地址范围 0x40000000 ~ 0xBFFFFFFF），但当前编译产物烧录后，Linux 通过 `free -h` 仅能看到约 101MB 可用内存，未充分利用 2GB 物理内存。

### 1.1 当前问题表现

在开发板串口终端执行：

```
free -h
#           total        used        free        shared    buffers
# Mem:      101M          80M          22M           8.0K         1.4M

cat /proc/cmdline
# mem=132928K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5 ...
```

Linux 内核只使用了 `mem=132928K`（约 130MB），剩余 DDR 空间未分配给 Linux 或 MMZ。

### 1.2 根因分析

内存分配受三层配置限制，从底层到上层依次为：

| 层级          | 配置项                              | 当前值                  | 问题                         |
|-------------|----------------------------------|----------------------|----------------------------|
| DDR 物理初始化   | reg_info.bin（BL2 阶段）             | 2GB                  | 无问题，已正确配置                  |
| u-boot 环境变量 | emmc_env.bin → bootargs mem=     | 132928K (~130MB)     | <b>直接瓶颈</b> ，限制 Linux 可见内存 |
| 内核设备树       | hi3519dv500-demb.dts → memory 节点 | reg = 1GB            | 声明的内存上限不足 2GB              |
| MPP 媒体内存    | load3519dv500 → MMZ 参数           | mmz_start + mmz_size | 需与 mem= 值联动调整              |

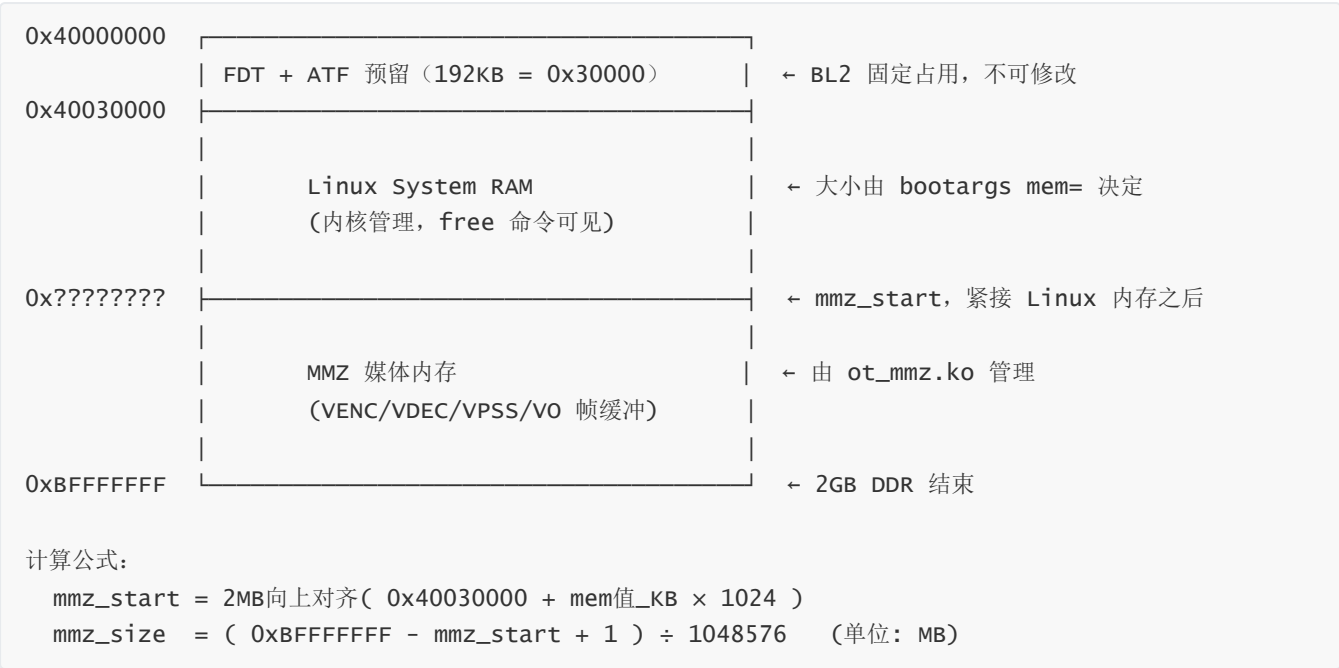
### 1.3 为什么不需要修改 u-boot

u-boot 的 `PHYS_SDRAM_1_SIZE = 0x20000000`（512MB）**不需要修改**，原因：

- reg\_info.bin 已完成 DDR 物理初始化：** BL2 阶段由 `reg_info.bin`（源自 Hi3519DV500-DMEB\_6L\_T\_DDR4\_2666M-2GB\_16bitx2-A55\_1G.xlsm）负责 DDR 控制器训练，到 u-boot 启动时 2GB DDR 已全部可用
- u-boot 只用前 512MB 就够了：** bootcmd 中加载 BL31 地址 `0x40010000`、内核地址 `0x44000000`，均在 512MB 范围内
- Linux 内存不由 u-boot 的宏决定：** 由 bootargs `mem=` 和 DTS `memory` 节点控制

- **MMZ 由内核模块运行时映射：** `ot_mmz.ko` 通过 `ioremap` 直接映射物理内存，绕过 u-boot
- 唯一副作用： `bdinfo` 显示 DRAM bank size = 512MB（信息不准），不影响功能

## 2. DDR 物理内存布局



## 3. 三种分配方案

| 方案 | emmc_env.txt mem= | Linux 内存 | mmz_start  | mmz_size | MMZ 内存 |
|----|-------------------|----------|------------|----------|--------|
| C  | 132928K           | ~130MB   | 0x48200000 | 1918M    | 1918MB |
| B  | 262144K           | 256MB    | 0x50200000 | 1790M    | 1790MB |
| A  | 524288K           | 512MB    | 0x60200000 | 1534M    | 1534MB |

选择建议：

- **IP Camera 场景 (推荐方案 B 或 C)：** MMZ 需要大量内存用于视频编解码帧缓冲，Linux 本身不需要太多内存。方案 C 给 Linux 130MB、MMZ 1918MB；方案 B 给 Linux 256MB 更宽裕
- **通用 Linux 场景 (方案 A)：** 如果主要跑用户态程序而非媒体处理，给 Linux 512MB 更合适

## 4. 需要修改的文件清单

以下所有路径均为相对于项目根目录的相对路径。

| # | 文件路径                                                                                                                      | 修改内容                    | 是否必须 |
|---|---------------------------------------------------------------------------------------------------------------------------|-------------------------|------|
| 1 | device/soc/hisilicon/hi3519dv500/sdk_linux/smp/a55_linux/source/bsp/tools/pc/u-boot_env/env_text/hi3519dv500/emmc_env.txt | bootargs 的 mem= 参数      | 必须   |
| 2 | device/soc/hisilicon/hi3519dv500/burn/emmc_env.bin                                                                        | 由 emmc_env.txt 重新生成的二进制 | 必须   |
| 3 | kernel/linux/patches/linux-5.10/hisilicon/hi3519dv500_patch/oh_i9dv500.patch                                              | DTS memory 节点 reg 值     | 必须   |
| 4 | device/soc/hisilicon/hi3519dv500/sdk_linux/smp/a55_linux/source/out/ko/load3519dv500                                      | mmz_start 和 mmz_size    | 必须   |

## 5. 详细修改步骤

### 5.1 修改 emmc\_env.txt (u-boot 环境变量源文件)

文件路径：

```
device/soc/hisilicon/hi3519dv500/sdk_linux/smp/a55_linux/source/bsp/tools/pc/u-boot_env/env_t  
ext/hi3519dv500/emmc_env.txt
```

**为什么修改：**这个文件是 u-boot 环境变量的文本源文件，其中的 `bootargs` 包含 `mem=` 参数，直接决定 Linux 内核管理多少物理内存。当前预编译的 `emmc_env.bin` 中是 `mem=132928K` (130MB)，是 Linux 只能看到 101MB 的直接原因。

**当前内容** (注意：此文件已有 `mem=262144K`，但 burn 目录下的预编译 bin 仍是旧的 `mem=132928K`)：

```
bootargs=mem=262144K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5  
rootfstype=ext4 rw rootwait  
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)
```

**修改方式：**将 `mem=` 后的值改为目标方案对应的值。

以方案 A (512MB Linux) 为例，修改为：

```
bootargs=mem=524288K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5  
rootfstype=ext4 rw rootwait  
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)
```

以方案 B (256MB Linux) 为例，修改为：

```
bootargs=mem=262144K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5  
rootfstype=ext4 rw rootwait  
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)
```

**注意：**此文件同时还定义了 eMMC 分区表 (`blkdevparts=mmcblk0:...`)。当前 SDK 源码版本已有 `rootfs=200M,userdata=50M` (比预编译 bin 的 `rootfs=96M,userdata=10M` 更大)，采用此文件的配置即可。

### 5.2 重新生成 emmc\_env.bin

**为什么重新生成：**`device/soc/hisilicon/hi3519dv500/burn/emmc_env.bin` 是预编译的二进制文件，SDK 构建脚本 (`sdk_linux/build.sh`) 会直接把它拷贝到编译输出目录。当前预编译 bin 仍是旧的 `mem=132928K` 版本，必须用修改后的 `emmc_env.txt` 重新生成。

**生成方法** (在容器内执行)：

```
cd device/soc/hisilicon/hi3519dv500/sdk_linux/smp/a55_linux/source/bsp/tools/pc/u-boot_env/

# 使用 mkenvimage 工具生成 env.bin
make clean
make ENVTEXT=env_text/hi3519dv500/emmc_env.txt SIZE=0x40000

# 将生成的 env.bin 拷贝到 burn 目录，替换旧的预编译文件
cp env.bin ../../../../burn/emmc_env.bin

# 同时用 env2burn.py 重新生成烧录分区表
python3 scripts/env2burn.py env_text/hi3519dv500/emmc_env.txt
../../../../burn/emmc_burn_table.xml
```

验证生成的文件：

```
strings burn/emmc_env.bin | grep mem=
# 应输出: bootargs=mem=524288K ... (与你设置的值一致)
```

关键参数说明：

- `SIZE=0x40000`：env 分区大小 256KB，与 burn\_table.xml 中 env 分区 `Length="0x80000"`（512KB，含校验头）对应
- `mkenvimage`：u-boot 提供的工具，将文本 env 转换为 u-boot 可识别的二进制格式（含 CRC32 校验头）

## 5.3 修改内核 DTS memory 节点

文件路径：

```
kernel/linux/patches/linux-5.10/hispark_hi3519dv500_patch/oh_19dv500.patch
```

为什么修改这个 patch 文件而不是直接修改 DTS 文件：

内核构建流程（`kernel/linux/build/kernel.mk`）每次编译都会：

1. `rm -rf $(KERNEL_SRC_TMP_PATH)` — 彻底删除临时构建目录
2. `cp -arfl $(KERNEL_SRC_PATH)/ * → 临时目录` — 复制干净内核源码
3. 执行 `patch_hi3519dv500.sh` — 打补丁，其中 `oh_19dv500.patch` 以 `new file` 方式创建 `hi3519dv500-demb.dts`

`hi3519dv500-demb.dts`（包含 memory 节点的文件）**不存在于源码树中**，只在构建输出的 `out/` 目录中，每次全量编译由 patch 重新生成。因此必须修改 patch 文件。

源码树中实际存在的 DTS 文件只有三个（不包含 memory 节点）：

```
device/soc/hisilicon/hi3519dv500/kernel/arch/arm64/boot/dts/vendor/
├── hi3519dv500-demb-emmc.dts    # eMMC 变体, include "hi3519dv500-demb.dts"
├── hi3519dv500-demb-flash.dts  # Flash 变体
└── hi3519dv500.dtsi           # SoC 外设定义
```

`DEVICE_DTS_PATH` 变量在 `kernel.mk` 中虽然被定义，但搜索整个文件只有赋值语句 `:=`，没有任何引用使用——是死代码，不会把 `device/soc/` 下的 DTS 复制到构建目录。

#### 修改方法：

在 `oh_19dv500.patch` 中搜索 `hi3519dv500-demb.dts`（约第 23139 行开始），找到 `memory` 节点定义段：

修改前（第 23211 行附近）：

```
+     memory {
+         device_type = "memory";
+         reg = <0x0 0x40030000 0x0 0x40000000>;
+     };
```

修改后（声明 2GB，扣除 FDT&ATF 的 192KB）：

```
+     memory {
+         device_type = "memory";
+         reg = <0x0 0x40030000 0x0 0x7FFD0000>;
+     };
```

#### 参数计算：

```
reg 第三个字（size 字段） = DDR 总大小 - FDT&ATF 预留
                          = 0x80000000 - 0x30000
                          = 0x7FFD0000
                          ≈ 2047MB
```

DTS `memory` 节点声明的是物理内存的上限，`mem=` `bootargs` 参数进一步限制 Linux 实际使用的内存。两者都需要修改为支持 2GB。

**注意：**同一段中还有 `chosen` 节点的 `bootargs` 默认值 `mem=512M`，虽然会被 u-boot env 覆盖，但建议保持一致性，改为与 `emmc_env.txt` 中相同的值。

## 5.4 修改 load3519dv500 脚本（MMZ 参数）

#### 文件路径：

```
device/soc/hisilicon/hi3519dv500/sdk_linux/smp/a55_linux/source/out/ko/load3519dv500
```

**为什么修改：**这个脚本负责加载 MPP 媒体驱动模块，其中 `mmz_start` 和 `mmz_size` 参数传给 `ot_mmz.ko`，决定媒体处理（VENC/VDEC/VPSS/VO 等）可用的帧缓冲内存。`mmz_start` 必须紧接在 Linux 内存之后，`mmz_size` 不能超过 DDR 末端，两者必须与 `mem=` 值联动。

**当前内容（第 33-40 行）：**

```
# DDR start:      0x40000000
# 192K, FDT&ATF mem
# kernel start: 0x40030000
# 132928K(1856K+128M), os mem
# MMZ start:      0x48200000
# 1918M, mmz mem
```

```
mmz_start=0x48200000;
mmz_size=1918M;
```

### 参数计算推导：

1. Linux 内存结束地址（原始值）：  
$$\text{raw\_end} = 0x40030000 + \text{mem\_KB} \times 1024$$
2. mmz\_start 需要向上对齐到 2MB 边界（海思 MMZ DMA 对齐要求）：  
$$\text{mmz\_start} = ((\text{raw\_end} + 0x1FFFFFF) \div 0x200000) \times 0x200000$$
3. mmz\_size = DDR 末端地址 - mmz\_start + 1, 转换为 MB：  
$$\text{mmz\_size} = (0xBFFFFFFF - \text{mmz\_start} + 1) \div 0x100000$$

### 当前配置的验证：

```
raw_end    = 0x40030000 + 132928 × 1024 = 0x40030000 + 0x81B7000 = 0x481B7000
mmz_start  = 对齐后 0x48200000 ✓（与脚本一致）
mmz_size   = (0xBFFFFFFF - 0x48200000 + 1) ÷ 0x100000 = 0x77E00000 ÷ 0x100000 = 1918 ✓
```

### 各方案修改内容：

#### 方案 C（保持 mem=132928K，仅扩大到 2GB）

不需要修改此文件，参数已经是 2GB DDR 下 mem=132928K 的正确值。

#### 方案 B（mem=262144K，Linux 256MB）

```
# DDR start:      0x40000000
# 192K, FDT&ATF mem
# kernel start: 0x40030000
# 262144K(1856K+256M), os mem
# MMZ start:      0x50200000
# 1790M, mmz mem
```

```
mmz_start=0x50200000;
mmz_size=1790M;
```

### 计算验证：

```
raw_end    = 0x40030000 + 262144 × 1024 = 0x50030000
mmz_start  = 0x50200000（2MB 对齐向上）
mmz_size   = (0xBFFFFFFF - 0x50200000 + 1) ÷ 0x100000 = 0x6FE00000 ÷ 0x100000 = 1790
```

## 方案 A (mem=524288K, Linux 512MB)

```
# DDR start:    0x40000000
# 192K, FDT&ATF mem
# kernel start: 0x40030000
# 524288K(1856K+512M), os mem
# MMZ start:    0x60200000
# 1534M, mmz mem
```

```
mmz_start=0x60200000;
mmz_size=1534M;
```

计算验证:

```
raw_end    = 0x40030000 + 524288 × 1024 = 0x60030000
mmz_start  = 0x60200000 (2MB 对齐向上)
mmz_size   = (0xBFFFFFFF - 0x60200000 + 1) ÷ 0x100000 = 0x5FE00000 ÷ 0x100000 = 1534
```

参数生效位置 (第 106 行) :

```
insmod ot_mmz.ko anony=1 mmz_allocator=ot mmz=anonymous,0,$mmz_start,$mmz_size$1
mem_process_isolation=1
```

展开后实际执行:

```
insmod ot_mmz.ko anony=1 mmz_allocator=ot mmz=anonymous,0,0x50200000,1790M
mem_process_isolation=1
```

\$1 在主流程调用 `insert_ko` 时为空, 不影响参数。

关于 `load3519dv500_sec` 文件:

`load3519dv500_sec` 是安全启动 (TEE) 版本, 额外预留了 OPTEE (16MB) 和 SMMZ (64MB), MMZ 计算的 base 不同:

```
sec_base = 0x40030000 + 0x1000000(OPTEE) + 0x4000000(SMMZ) = 0x4D300000
```

如果使用安全启动, 需要用同样的公式但基于 `sec_base` 计算 `mmz_start` 和 `mmz_size`。

## 6. 编译和烧录

### 6.1 编译

修改完成后, 全量编译:

```
rm -rf out && hb build -f
```

全量编译确保:

- 内核 DTS 从 patch 重新生成 (反映 memory 节点修改)

- SDK KO 模块和 load 脚本从源码重新拷贝（反映 MMZ 参数修改）
- emmc\_env.bin 从 burn 目录拷贝到输出（反映 mem= 修改）

## 6.2 烧录

需要重新烧录的镜像（通过 HiTool 或 USB 烧录工具）：

| 分区     | 文件              | 修改原因                  |
|--------|-----------------|-----------------------|
| env    | emmc_env.bin    | 包含新的 mem= bootargs    |
| kernel | ulimage-fdt     | 包含新的 DTS memory 节点    |
| rootfs | rootfs_ext4.img | 包含新的 load3519dv500 脚本 |

## 7. 板端验证

### 7.1 验证 Linux 可见内存

```
free -h
# 方案 B 预期: total 约 256M
# 方案 A 预期: total 约 512M

cat /proc/cmdline
# 确认 mem= 值正确
```

### 7.2 验证 MMZ 媒体内存

```
# 加载驱动模块
cd /ko && ./load3519dv500 -i

# 检查 MMZ 是否成功申请
cat /proc/media-mem
# 应显示 MMZ 大小与 mmz_size 设置值一致
```

### 7.3 验证 DDR 总使用量

```
cat /proc/iomem | grep "System RAM"
# 应显示从 0x40030000 开始，到与 mem= 对应的结束地址

# Linux System RAM + MMZ 的总和应约等于 2GB
```

### 7.4 运行时快速测试（不改文件）

在修改源文件之前，可以先在板端通过命令行参数验证 MMZ 配置是否正确：

```
# 以方案 B 为例
./load3519dv500 -i -mmz_start 0x50200000 -mmz_size 1790M
```



如果 `insmod ot_mmz.ko` 成功且 `/proc/media-mem` 显示正确大小，说明参数计算正确，可以放心修改源文件。

## 8. 快速验证参数计算 (Python 脚本)

可在容器内运行以下命令验证任意 `mem=` 值对应的 MMZ 参数：

```
python3 -c "  
ddr_end = 0xBFFFFFFF  
base = 0x40030000  
mem_k = int(input('输入 mem= 值 (单位 KB): '))  
raw = base + mem_k * 1024  
aligned = ((raw + 0x1FFFFFF) // 0x200000) * 0x200000  
size = (ddr_end - aligned + 1) // 0x100000  
print(f'mmz_start=0x{aligned:08x}')  
print(f'mmz_size={size}M')  
"
```

## 9. 注意事项

1. **mem=、mmz\_start、mmz\_size 必须联动修改**：三者是关联的，单独修改任何一个都会导致内存布局不一致
2. **emmc\_env.txt 和 emmc\_env.bin 必须同步**：修改 txt 后必须重新生成 bin，否则烧录的仍是旧配置
3. **DTS memory 节点的 reg size 是上限声明**：它必须  $\geq$  mem= 指定的实际大小，否则内核启动时可能只识别到 DTS 声明的范围
4. **mmz\_start 的 2MB 对齐**：由于 FDT&ATF 预留了 192KB (0x30000)，当 mem= 为整数 MB 值时，mmz\_start 会有约 1.8MB 的对齐间隙，这是正常的，不影响功能
5. **不要修改 reg\_info.bin 或 boot\_image.bin**：DDR 物理初始化已正确配置为 2GB，无需修改
6. **增量编译不保证 DTS 更新生效**：建议全量编译 `rm -rf out && hb build -f`